



BASES DE DONNÉES

CM SQL avancé :

Tables temporaires

Tables dérivées

Vincent COUTURIER

Maitre de conférences – Université Savoie Mont-Blanc

vincent.couturier@univ-smb.fr

Préambule

- Schéma de la base de données EMPLOYEES :
 - DEPT (DEPTNO, DNAME, LOC)
 - EMP (EMPNO, ENAME, ADR, JOB, #MGR, HIREDATE, SAL, COMM, #DEPTNO)

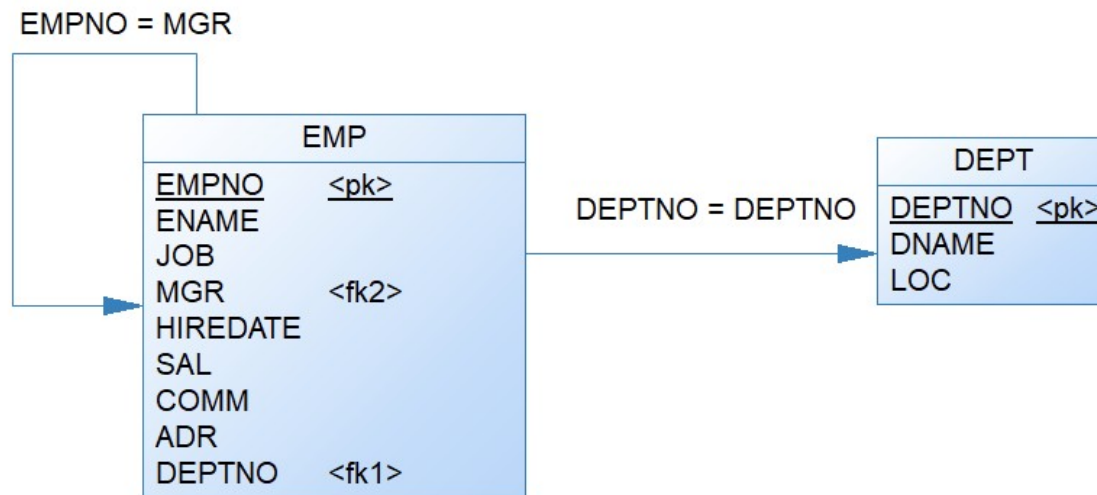


TABLE TEMPORAIRE

Table temporaire ?

- Table qui a une existence – et donc n'est utilisable – que pour la session (connexion) qui l'a créée et sera supprimée une fois la session terminée (ou au choix une fois la transaction validée).
- Les tables temporaires sont utiles lors du traitement des données, notamment lors d'une transformation lorsque les résultats intermédiaires sont temporaires => Utilisées pour réaliser des calculs (ou requêtes) complexes.
- La table temporaire peut être créée par du code SQL, par une application (C#, PHP,...) qui l'utilisera à des fins de calculs ou par du code procédural stocké dans la base de données (procédures ou fonctions stockées).
- Les tables temporaires offrent un gain de performances, car leurs résultats sont écrits en local et non dans un stockage distant.
- Les tables temporaires ne sont pas visualisables dans le schéma *public* (elles sont stockées dans un autre schéma). MAIS le nom de la table temporaire doit être distinct du nom des autres objets du schéma (tables, séquences, vues, etc.).

Syntaxe partielle

```
CREATE TEMPORARY TABLE [ IF NOT EXISTS ] nom_table  
    [ (nom_colonne [, ...] ) ]  
    [ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]  
    AS requête
```

- On peut écrire CREATE TEMPORARY TABLE... ou CREATE TEMP TABLE...
- **nom_table** : nom de la table à créer
- **nom_colonne** : nom d'une colonne dans la nouvelle table. Si les noms de colonnes ne sont pas précisés, ils sont issus des noms des colonnes en sortie de la requête.
- ON COMMIT
 - Le comportement des tables temporaires à la fin d'un bloc de transaction est contrôlable en utilisant ON COMMIT. Voici les trois options :
 - PRESERVE ROWS : aucune action spéciale n'est effectuée à la fin de la transaction. C'est le comportement par défaut (et qui est généralement utilisé)
 - DELETE ROWS : toutes les lignes de la table temporaire seront supprimées à la fin de la transaction.
 - DROP : la table temporaire sera supprimée à la fin du bloc de transaction en cours.

Exemple d'utilisation

- Table temporaire qui stocke les moyennes de salaires par département :

```
DROP TABLE IF EXISTS temp_moyennesalaires_departement;
```

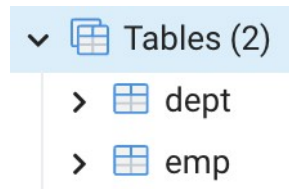
```
CREATE TEMP TABLE temp_moyennesalaires_departement AS  
  SELECT d.deptno as NumDept, d.dname as NomDept,  
         ROUND(AVG(e.sal),2) as MoyenneSalaire  
  FROM dept d  
        JOIN emp e ON d.deptno=e.deptno  
 GROUP BY d.deptno, d.dname;
```

```
SELECT * FROM temp_moyennesalaires_departement;
```

	numdept numeric (2)	nomdept character varying (14)	moyennesalaire numeric
1	30	SALES	1566.67
2	10	ACCOUNTING	2916.67
3	20	RESEARCH	2175.00

Exemple d'utilisation

- La table temporaire n'est pas visualisable dans la liste des tables sous PgAdmin 4 :



- Pour visualiser les tables temporaires :

```
SELECT table_name
FROM information_schema.tables
WHERE table_type='LOCAL TEMPORARY';
```

	table_name	
	name	
1	t_moyennesalaires_departement	

Information_schema est le catalogue système de PostgreSQL. Il contient des méta-données (informations) sur tout ce que vous avez créé. Dans ce catalogue, se trouve l'objet système tables contient des informations sur toutes les tables que vous avez créées.

Exemple d'utilisation

- Classement des départements par moyenne de salaires décroissante

```
SELECT NumDept, NomDept, MoyenneSalaire,  
       RANK() OVER (  
           ORDER BY MoyenneSalaire DESC) AS Classement  
FROM temp_moyennesalaires_departement;
```

	numdept numeric (2) 🔒	nomdept character varying (14) 🔒	moyennesalaire numeric 🔒	classement bigint 🔒
1	10	ACCOUNTING	2916.67	1
2	20	RESEARCH	2175.00	2
3	30	SALES	1566.67	3

Remarques : fonctions RANK() et DENSE_RANK()

- rank() : rang de la ligne courante. Les lignes identiques ont le même rang, MAIS avec des valeurs manquantes (trous) dans le classement. Ex. : 1, 2, 2, 4
- dense_rank() : rang de la ligne courante. Les lignes identiques ont le même rang, MAIS sans valeur manquante dans le classement. Ex. : 1, 2, 2, 3
- Fonctions fenêtrées (*Window functions*) qui ressemblent aux agrégats et aux regroupements (dans leur version simple).
- Dans leur version simple, elles utilisent la clause OVER obligatoire et s'appliquent sur tous les enregistrements de la requête.
 - On pourra retrouver dans cette clause OVER un tri réalisé avec ORDER BY.
- Il existe un mot clé optionnel (clause PARTITION BY) qui permet d'appliquer la fonction fenêtrée sur chaque sous-ensemble des enregistrements de la requête. Ce sous-ensemble est appelé une *fenêtre*.

Principales autres fonctions fenêtrées

- `row_number ( )` : n° de ligne de chaque enregistrement du jeu de résultat. Ex. : 1, 2, 3, 4
- `percent_rank ( )` : rang relatif de la ligne courante en pourcentage.
 - La fonction utilise le calcul suivant : $(\text{rang} - 1) / (\text{rang max} - 1)$
- `cume_dist ( )` : calcule la distribution cumulée (percentile). Pour un employé, cela représente la fraction de lignes dont la valeur est inférieure ou égale à la sienne.
 - Utile pour identifier les Top N% du jeu de données (ex: top 10% des salaires)
- `ntile(n)` : partitionnement du jeu de résultat en n groupes (seaux) égaux
- `first_value(attribut)` : retourne la première valeur du jeu de résultat pour l'*attribut*.

Exemple n°1

```
SELECT empno, ename, sal,  
       rank() OVER (ORDER BY sal DESC) AS Rang,  
       dense_rank() OVER (ORDER BY sal DESC) AS RangDense,  
       percent_rank() OVER (ORDER BY sal DESC) AS RangEnPourcent,  
       cume_dist() OVER (ORDER BY sal DESC) AS RangEnFraction,  
       row_number() OVER (ORDER BY sal DESC) AS Numligne,  
       ntile(3) OVER (ORDER BY sal DESC) AS NumGroupe  
FROM emp;
```

Exemple n°1

	empno numeric (4)	ename character varying (10)	sal numeric (7,2)	rang bigint	rangdense bigint	rangenpourcent double precision	rangenfraction double precision	numligne bigint	numgroupe integer
1	7839	KING	5000.00	1	1	0	0.07142857142857142	1	1
2	7902	FORD	3000.00	2	2	0.07692307692307693	0.21428571428571427	2	1
3	7788	SCOTT	3000.00	2	2	0.07692307692307693	0.21428571428571427	3	1
4	7566	JONES	2975.00	4	3	0.23076923076923078	0.2857142857142857	4	1
5	7698	BLAKE	2850.00	5	4	0.3076923076923077	0.35714285714285715	5	1
6	7782	CLARK	2450.00	6	5	0.38461538461538464	0.42857142857142855	6	2
7	7499	ALLEN	1600.00	7	6	0.46153846153846156	0.5	7	2
8	7844	TURNER	1500.00	8	7	0.5384615384615384	0.5714285714285714	8	2
9	7934	MILLER	1300.00	9	8	0.6153846153846154	0.6428571428571429	9	2
10	7654	MARTIN	1250.00	10	9	0.6923076923076923	0.7857142857142857	10	2
11	7521	WARD	1250.00	10	9	0.6923076923076923	0.7857142857142857	11	3
12	7876	ADAMS	1100.00	12	10	0.8461538461538461	0.8571428571428571	12	3
13	7900	JAMES	950.00	13	11	0.9230769230769231	0.9285714285714286	13	3
14	7369	SMITH	800.00	14	12	1	1	14	3

- Num groupe : 3 groupes car `ntile(3)`. Si on désire faire 4 quarts (quartile), ce sera `ntile(4)`.
- Rang en fraction (`cume_dist`) : $0,071=1/14$ (14=nb de lignes). Pourcentage d'employés dont le salaire est supérieur ou égal à celui de l'employé actuel. Percentile des salaires.
- Rang en pourcent (`percent_rank`) : $(rang-1)/(rang\ max -1)$. Rang du salaire de l'employé sous forme de pourcentage : $0,0769=1/13$

Exemple n°2

```
SELECT empno, ename, sal,
       first_value(sal) OVER (ORDER BY sal DESC)
         as "salaire le + haut",
       first_value(sal) OVER (ORDER BY sal ASC)
         as "salaire le + bas",
       sal - first_value(sal) OVER (ORDER BY sal ASC)
         as "Difference/salaire le + bas"
FROM emp
```

	empno numeric (4)	ename character varying (10)	sal numeric (7,2)	salaire le + haut numeric	salaire le + bas numeric	Difference/salaire le + bas numeric
1	7369	SMITH	800.00	5000.00	800.00	0.00
2	7900	JAMES	950.00	5000.00	800.00	150.00
3	7876	ADAMS	1100.00	5000.00	800.00	300.00
4	7521	WARD	1250.00	5000.00	800.00	450.00
5	7654	MARTIN	1250.00	5000.00	800.00	450.00
6	7934	MILLER	1300.00	5000.00	800.00	500.00
7	7844	TURNER	1500.00	5000.00	800.00	700.00
8	7499	ALLEN	1600.00	5000.00	800.00	800.00
9	7782	CLARK	2450.00	5000.00	800.00	1650.00
10	7698	BLAKE	2850.00	5000.00	800.00	2050.00
11	7566	JONES	2975.00	5000.00	800.00	2175.00
12	7788	SCOTT	3000.00	5000.00	800.00	2200.00
13	7902	FORD	3000.00	5000.00	800.00	2200.00
14	7839	KING	5000.00	5000.00	800.00	4200.00

TABLES DERIVEES OU VUES EN LIGNE

Tables dérivées

- Une *table dérivée* (aussi appelée *vue en ligne*) est considérée comme une table ou une vue temporaire qui n'existe que pour une requête.
- Ainsi, le classement des départements par moyenne de salaires décroissante peut se faire en une seule étape, SANS avoir besoin de vue SQL ni de table temporaire :

```
SELECT t.NumDept, t.NomDept, t.MoyenneSalaire,
       RANK () OVER (ORDER BY t.MoyenneSalaire DESC) AS Classement
FROM
    (SELECT d.deptno as NumDept, d.dname as NomDept,
    ROUND(AVG(e.sal),2) as MoyenneSalaire
    FROM dept d
        JOIN emp e ON d.deptno=e.deptno
    GROUP BY d.deptno, d.dname) AS t;
```

Table dérivée qui
se nommera t

	numdept numeric (2)	nomdept character varying (14)	moyennesalaire numeric	classement bigint
1	10	ACCOUNTING	2916.67	1
2	20	RESEARCH	2175.00	2
3	30	SALES	1566.67	3

Tables dérivées

- Il est possible d'imbriquer une table dérivée dans une autre table dérivée.
- **Autre écriture (A PREFERER POUR LA FACILITE DE LECTURE) avec une requête CTE :**

```
WITH t AS (
    SELECT d.deptno as NumDept, d.dname as NomDept,
    ROUND(AVG(e.sal),2) as MoyenneSalaire
    FROM dept d
        JOIN emp e ON d.deptno=e.deptno
    GROUP BY d.deptno, d.dname
)
SELECT t.NumDept, t.NomDept, t.MoyenneSalaire,
    RANK () OVER (ORDER BY t.MoyenneSalaire DESC) AS Classement
FROM t;
```

- Il est possible d'imbriquer des blocs WITH dans d'autres blocs WITH

CTE

- Les **Common Table Expressions** (CTE) remplacent avantageusement les vues en ligne ou tables dérivées.
 - **Lisibilité** : Structure la requête de haut en bas.
 - **Maintenance** : Facilite le débogage en isolant les sous-calculs.
 - **Réutilisabilité** : On peut référencer le bloc WITH plusieurs fois dans la même requête

Tables dérivées

- Les tables dérivées et CTE ne sont à utiliser que si vous avez des requêtes complexes à écrire, par exemple, un calcul (fonction) qui porte sur un autre calcul (fonction d'agrégat).
- Dans le cas de requêtes simples, il faut privilégier l'écriture normalisée :
SELECT... FROM... WHERE... GROUP BY... HAVING... ORDER BY...

```

1  SELECT d.deptno as NumDept, d.dname,
2      ROUND(AVG(e.sal),2) as MoyenneSalaire
3  FROM dept d
4      JOIN emp e ON d.deptno=e.deptno
5  GROUP BY d.deptno, d.dname
6  HAVING ROUND(AVG(e.sal),2)>=2000;
7
8  WITH t AS (
9      SELECT d.deptno, d.dname, ROUND(AVG(e.sal),2) as MoyenneSalaire
10     FROM dept d
11         JOIN emp e ON d.deptno=e.deptno
12     GROUP BY d.deptno, d.dname)
13  SELECT t.*
14  FROM t
15  WHERE t.MoyenneSalaire>=2000
16

```

Données EXPLAIN Messages Notifications

	numdept numeric (2)	dname character varying (14)	moyennesalaire numeric	
1	10	ACCOUNTING	2916.67	
2	20	RESEARCH	2175.00	

Table temporaire vs. table dérivée vs. vue ?

- **En principe, on n'utilise pas une vue si on a un besoin ponctuel. Il est préférable d'utiliser une table temporaire ou une table dérivée**

REQUETE RECURSIVE

Naviguer entre les hiérarchies

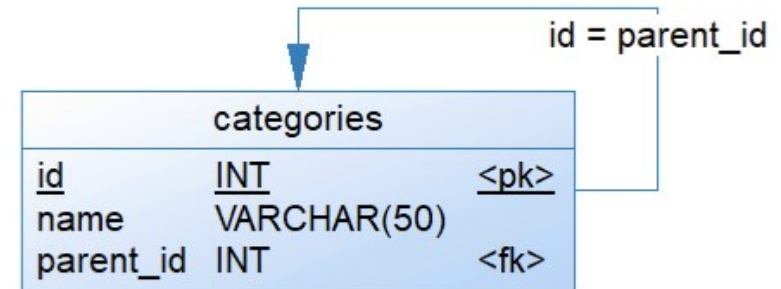
- **WITH RECURSIVE**
- Permet de parcourir des arbres de catégories ou des organigrammes (MGR / EMPNO).
- Se compose d'un **point de départ** (ancree) et d'une **partie réursive** jointe à l'ancree via UNION ALL.
- Idéal pour calculer des niveaux de profondeur (Level) ou des chemins de dossiers.

Exemple

- Modèle :

```

DROP TABLE IF EXISTS categories;
CREATE TABLE categories (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    parent_id INT,
    constraint FK_categories_parent_id FOREIGN KEY (parent_id) REFERENCES categories(id)
);
INSERT INTO categories (id, name, parent_id) VALUES
(1, 'Tous les produits', NULL),
(2, 'Électronique', 1),
(3, 'Informatique', 2),
(4, 'Périphériques', 3),
(5, 'Claviers', 4),
(6, 'Souris', 4),
(7, 'Écrans', 3),
(8, 'Clavier Azerty', 5),
(9, 'Clavier Querty', 5),
(10, 'Clavier Quertz', 5),
(11, 'Téléphones', 2),
(12, 'Mac', 3),
(13, 'TV', 2),
(14, 'PC', 3),
(15, 'TV 4K', 13);
  
```



Exemple

- Requête récursive pour remonter la hiérarchie à partir de la catégorie "Claviers"

```
WITH RECURSIVE categories_tree AS (  
    SELECT id, name, parent_id  
    FROM categories  
    WHERE name = 'Claviers'  
    UNION ALL  
    SELECT c.id, c.name, c.parent_id  
    FROM categories c  
    JOIN categories_tree ct ON c.id = ct.parent_id  
)  
SELECT * FROM categories_tree;
```

id integer	name character varying (50)	parent_id integer
5	Claviers	4
4	Périphériques	3
3	Informatique	2
2	Électronique	1
1	Tous les produits	[null]

Exemple

- Requête récursive pour remonter la hiérarchie à partir de la catégorie "Clavier Quartz"

```
WITH RECURSIVE categories_tree AS (
    SELECT id, name, parent_id
    FROM categories
    WHERE name = 'Clavier Quartz'
    UNION ALL
    SELECT c.id, c.name, c.parent_id
    FROM categories c
    JOIN categories_tree ct ON c.id = ct.parent_id
)
SELECT * FROM categories_tree;
```

id integer	name character varying (50)	parent_id integer
10	Clavier Quartz	5
5	Claviers	4
4	Périphériques	3
3	Informatique	2
2	Électronique	1
1	Tous les produits	[null]

Exemple

- Requête récursive pour descendre toute la hiérarchie

```
WITH RECURSIVE categories_tree AS (  
    -- Point de départ : la catégorie mère  
    SELECT id, name, parent_id  
    FROM categories  
    WHERE parent_id IS NULL  
    UNION ALL  
    -- Partie récursive : on va chercher les enfants  
    SELECT c.id, c.name, c.parent_id  
    FROM categories c  
    JOIN categories_tree ct ON c.parent_id = ct.id  
)  
SELECT * FROM categories_tree;
```

Exemple

- Requête récursive pour descendre toute la hiérarchie

id integer	name character varying (50)	parent_id integer
1	Tous les produits	[null]
2	Électronique	1
3	Informatique	2
11	Téléphones	2
13	TV	2
4	Périphériques	3
7	Écrans	3
12	Mac	3
14	PC	3
15	TV 4K	13
5	Claviers	4
6	Souris	4
8	Clavier Azerty	5
9	Clavier Querty	5
10	Clavier Quartz	5

Exemple

- Requête récursive pour descendre toute la hiérarchie avec ajout d'un niveau

```
WITH RECURSIVE categories_tree AS (  
    -- Point de départ : niveau 0  
    SELECT id, name, parent_id, 0 AS level  
    FROM categories  
    WHERE parent_id IS NULL  
    UNION ALL  
    -- Partie récursive : on descend vers les enfants, on  
    incrémente le niveau  
    SELECT c.id, c.name, c.parent_id, ct.level + 1  
    FROM categories c  
    JOIN categories_tree ct ON c.parent_id = ct.id  
)  
SELECT * FROM categories_tree  
ORDER BY level;
```

Exemple

- Requête récursive pour descendre toute la hiérarchie avec ajout d'un niveau

id integer	name character varying (50)	parent_id integer	level integer
1	Tous les produits	[null]	0
2	Électronique	1	1
3	Informatique	2	2
11	Téléphones	2	2
13	TV	2	2
4	Périphériques	3	3
7	Écrans	3	3
12	Mac	3	3
14	PC	3	3
15	TV 4K	13	3
5	Claviers	4	4
6	Souris	4	4
8	Clavier Azerty	5	5
9	Clavier Querty	5	5
10	Clavier Quertz	5	5

Exemple

- Affichage de la hiérarchie sous la forme d'un arbre

```
WITH RECURSIVE categories_tree AS (
  -- Point de départ : la catégorie racine (niveau 0)
  SELECT
    id, name, parent_id, 0 AS level,
    CAST(name AS VARCHAR(255)) AS affichage_arbre,
    ARRAY[id] AS chemin_tri -- Tableau pour stocker le chemin parcouru
  FROM categories
  WHERE parent_id IS NULL
  UNION ALL
  -- Partie récursive : on descend vers les enfants
  SELECT c.id, c.name, c.parent_id, ct.level + 1,
    CAST(REPEAT('  ', ct.level) || '|-- ' || c.name AS VARCHAR(255)),
    ct.chemin_tri || c.id -- On ajoute l'ID courant au tableau
  FROM categories c
  JOIN categories_tree ct ON c.parent_id = ct.id
)
SELECT id, affichage_arbre
FROM categories_tree
ORDER BY chemin_tri; -- Le tri par tableau permet de garder les branches groupées
```

Exemple

- Affichage de la hiérarchie sous la forme d'un arbre

	123 id	A-Z affichage_arbre
1	1	Tous les produits
2	2	-- Électronique
3	3	-- Informatique
4	4	-- Périphériques
5	5	-- Claviers
6	8	-- Clavier Azerty
7	9	-- Clavier Querty
8	10	-- Clavier Quartz
9	6	-- Souris
10	7	-- Écrans
11	12	-- Mac
12	14	-- PC
13	11	-- Téléphones
14	13	-- TV
15	15	-- TV 4K